

Week 1, Day 1 — Reference Notes

Git, GitHub, Project Structure & Troubleshooting

1. Repo Setup

Created a new repo on **GitHub** with an initial README, then cloned it locally:

```
git clone https://github.com/YOUR_USERNAME/data-analytics-course-2026.git
cd data-analytics-course-2026
code .
```

Checked connection status:

```
git status
```

→ Confirms you're on the right branch and connected to the remote.

2. Terminal Gotcha: PowerShell vs. Bash

- VSCode's default terminal on Windows may be **PowerShell**, which has different syntax than the **bash** commands used in most tutorials (e.g., `mkdir` in PowerShell doesn't accept multiple folder names the way bash does).
 - **Fix:** Switch VSCode's terminal to **Git Bash** via the terminal dropdown (▼ next to +) → **Select Default Profile** → Git Bash.
 - **Lesson:** Always know which shell you're in — commands aren't universal across shells.
-

3. `.gitignore` — What It Does and a Key Gotcha

Created `.gitignore` to tell Git which files/folders to never track:

```
# Environments
venv/
env/
.venv/
__pycache__/
*.pyc

# Jupyter
.ipynb_checkpoints/

# OS/editor junk
.DS_Store
.vscode/

# Data (large/raw files often shouldn't live in git)
data/raw/*
!data/raw/.gitkeep
```

Key gotcha learned: A rule like `data/raw/` ignores *everything* inside that folder — including placeholder files meant to keep the folder visible in Git. The fix is an **exception pattern**:

```
data/raw/*
!data/raw/.gitkeep
```

This ignores everything in the folder *except* the named file.

Lesson: `.gitignore` rules are broad by default; use `!` to carve out exceptions when you need a folder to exist in Git without tracking its contents.

4. Project Folder Structure

Created a clean, standard structure:

```
mkdir data notebooks src outputs
mkdir data/raw data/processed
touch data/raw/.gitkeep data/processed/.gitkeep outputs/.gitkeep
```

- `data/raw` — original/raw data (often excluded from Git)

- `data/processed` — cleaned/derived data
- `notebooks` — analysis notebooks
- `src` — reusable code/scripts
- `outputs` — results, figures, reports

Note: Git doesn't track empty folders — the `.gitkeep` trick (an empty placeholder file) keeps them visible in the repo.

5. Branch → Commit → Push → PR → Merge Cycle

This is the core professional Git workflow, practiced end-to-end:

```
# 1. Create and switch to a new branch
git checkout -b setup/project-structure

# 2. Stage changes
git add .

# 3. Commit with a clear message
git commit -m "Add project folder structure and .gitignore"

# 4. Push the branch to GitHub
git push -u origin setup/project-structure
```

Then on [GitHub.com](https://github.com):

- Click **Compare & pull request**
- Add a short title/description
- Click **Create pull request**, then **Merge pull request**

Finally, sync your local `main` branch:

```
git checkout main
git pull
```

Lesson: Working on a named branch (not directly on `main`) and going through a PR — even solo — builds the habit you'll need on real teams.

6. Debugging Workflow (This Is the Real Skill)

When `data/raw/.gitkeep` seemed to not get tracked, here's the systematic sequence used to diagnose it — a good general debugging pattern for Git issues:

1. Check if the file exists on disk:

```
ls -la data/raw
```

2. Check if it's staged:

```
git add data/raw/.gitkeep  
git status
```

3. Check if `.gitignore` is blocking it:

```
git check-ignore -v data/raw/.gitkeep
```

(No output = not being ignored.)

4. Force-add to rule out ignore issues:

```
git add -f data/raw/.gitkeep
```

5. Check if it's already tracked (the actual answer this time):

```
git ls-files | grep gitkeep
```

→ Revealed the file was already committed earlier — "nothing to commit" was correct all along, not a bug.

Lesson: When something seems broken but every check says "fine," consider that it might already be correct — verify assumptions (is it actually broken?) before assuming a config error.

Status at End of Day 1

✅ GitHub repo created and cloned ✅ Git Bash configured as terminal ✅ `.gitignore` created with proper exception handling ✅ Clean project folder structure in place and tracked ✅ First full

branch → PR → merge cycle completed successfully  Practiced real Git troubleshooting end-to-end

Next session: Step 3 — SQL from scratch using the Chinook database (joins, subqueries, window functions).